Assigment - 12.3

B.Varun
2303A52020
Batch - 38

Task 1: Sorting Student Records for Placement Drive
Scenario
SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.
Tasks
1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).
2. Implement the following sorting algorithms using AI assistance:
o Quick Sort
o Merge Sort
3. Measure and compare runtime performance for large datasets.
4. Write a function to display the top 10 students based on CGPA.
Expected Outcome
• Correctly sorted student records.
• Performance comparison between Quick Sort and Merge Sort.
• Clear output of top-performing students.

Code:

```
"""generate a program that stores student records
(Name, Roll Number, CGPA)."""
#student records
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __str__(self):
        return f"Name: {self.name}, Roll Number:
{self.roll_number}, CGPA: {self.cgpa}"
if __name__ == "__main__":
    student1 = Student("Alice", "001", 3.8)
    student2 = Student("Bob", "002", 3.5)
    student3 = Student("Charlie", "003", 3.9)
```

```
    print(student1)
    print(student2)
    print(student3)
```

```python
"""implementing quick sort and merge sort algorithms in
Python."""
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
if __name__ == "__main__":
    arr = [3, 6, 8, 10, 1, 2, 1]
    print("Original array:", arr)
    sorted_arr = quick_sort(arr)
    print("Quick Sort Sorted array:", sorted_arr)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
if __name__ == "__main__":
    arr = [38, 27, 43, 3, 9, 82, 10]
    print("Original array:", arr)
    sorted_arr = merge_sort(arr)
    print("Merge Sort Sorted array:", sorted_arr)
```

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/QuickSort.py"
Name: Alice, Roll Number: 001, CGPA: 3.8
Name: Bob, Roll Number: 002, CGPA: 3.5
Name: Charlie, Roll Number: 003, CGPA: 3.9
Original array: [3, 6, 8, 10, 1, 2, 1]
Quick Sort Sorted array: [1, 1, 2, 3, 6, 8, 10]
Original array: [38, 27, 43, 3, 9, 82, 10]
Merge Sort Sorted array: [3, 9, 10, 27, 38, 43, 82]
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task 2: Implementing Bubble Sort with AI Comments
• Task: Write a Python implementation of Bubble Sort.
• Instructions:
• Students implement Bubble Sort normally.
• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
• Request AI to provide time complexity analysis.
• Expected Output:
• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.

Code:

```
"""Implemnrtation of bubble sort algorithm in Python.
gemerating
in line comments explaining key logic (like
swapping, passes, and termination) and also providing the
time complexity of the algorithm."""
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to
check them
        for j in range(0, n-i-1):
            # Swap if the element found is greater than the
next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]  #
Swapping
                swapped = True  # Set the flag to True if
swapping occurs
        # If no swapping occurred, the array is already sorted
        if not swapped:
```

```
            break  # Terminate early if the array is already
sorted
    return arr
if __name__ == "__main__":
    arr = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr)
    sorted_arr = bubble_sort(arr)
    print("Sorted array:", sorted_arr)
# Time Complexity: O(n^2) in the worst and average case, O(n)
in the best case (when the array is already sorted).
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/BubbleSort.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task 3: Quick Sort and Merge Sort Comparison
• Task: Implement Quick Sort and Merge Sort using recursion.
• Instructions:
• Provide AI with partially completed functions for recursion.
• Ask AI to complete the missing logic and add docstrings.
• Compare both algorithms on random, sorted, and reverse-sorted lists.
• Expected Output:
• Working Quick Sort and Merge Sort implementations.
• AI-generated explanation of average, best, and worst-case complexities.

Code:

```
"""implementation of merge sort and quick sort algorithms in
Python using recursion. The program includes in-line comments
explaining the key logic of both algorithms, such as the
partitioning process in quick sort and the merging process in
merge sort.
1. Provide with partially completed functions for both
algorithms, and the user can fill in the missing parts to
complete the implementation.
2. complete the missing parts of the code to implement both
sorting algorithms correctly.
3. Compare both algorithms on random, sorted, and reverse-
sorted
lists to demonstrate their performance differences."""
```

```python
# partially completed quick sort function
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
# partially completed merge sort function
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
if __name__ == "__main__":
    # Test with random list
    arr_random = [3, 6, 8, 10, 1, 2, 1]
    print("Original random array:", arr_random)
    print("Quick Sort Sorted array:", quick_sort(arr_random))
    print("Merge Sort Sorted array:", merge_sort(arr_random))

    # Test with sorted list
    arr_sorted = [1, 2, 3, 4, 5]
    print("\nOriginal sorted array:", arr_sorted)
    print("Quick Sort Sorted array:", quick_sort(arr_sorted))
    print("Merge Sort Sorted array:", merge_sort(arr_sorted))
```

```python
    # Test with reverse-sorted list
    arr_reverse = [5, 4, 3, 2, 1]
    print("\nOriginal reverse-sorted array:", arr_reverse)
    print("Quick Sort Sorted array:", quick_sort(arr_reverse))
    print("Merge Sort Sorted array:", merge_sort(arr_reverse))
```

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppDat
a/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documen
ts/AI_Assisted_Coding/lab test-1/lab 9/lab 12/MergeSort.py"
Original random array: [3, 6, 8, 10, 1, 2, 1]
Quick Sort Sorted array: [1, 1, 2, 3, 6, 8, 10]
Merge Sort Sorted array: [1, 1, 2, 3, 6, 8, 10]

Original sorted array: [1, 2, 3, 4, 5]
Quick Sort Sorted array: [1, 2, 3, 4, 5]
Merge Sort Sorted array: [1, 2, 3, 4, 5]

Original reverse-sorted array: [5, 4, 3, 2, 1]
Quick Sort Sorted array: [1, 2, 3, 4, 5]
Merge Sort Sorted array: [1, 2, 3, 4, 5]
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task 4 (Real-Time Application – Inventory Management System)
Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:
1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.
Task:
• Use AI to suggest the most efficient search and sort algorithms for this use case.
• Implement the recommended algorithms in Python.
• Justify the choice based on dataset size, update frequency, and performance requirements.
Expected Output:
• A table mapping operation → recommended algorithm → justification.
• Working Python functions for searching and sorting the inventory.

Code:

```
"""
    Product class for managing inventory items.
    A Product represents an item in the inventory management
system with basic
    information and stock tracking capabilities.
    Attributes:
        product_id (int or str): Unique identifier for the
product.
        name (str): The name or title of the product.
        price (float): The price of the product.
        stock_quantity (int): The current quantity of the
product in stock.
    Methods:
        __init__(product_id, name, price, stock_quantity):
Initializes a new Product instance.
```

```
        __str__(): Returns a formatted string representation
of the product.
    """
"""include in-line comments explaining the key logic of the
class and its methods."""
"""using a best search algorithm for searching products by ID
or name, and a sorting algorithm like quicksort or mergesort
for sorting products by price or quantity."""
"""justify the chossen algorithm """
class Product:
    def __init__(self, product_id, name, price,
stock_quantity):
        self.product_id = product_id  # Unique identifier for
the product
        self.name = name  # Name of the product
        self.price = price  # Price of the product
        self.stock_quantity = stock_quantity  # Current stock
quantity

    def __str__(self):
        return f"Product ID: {self.product_id}, Name:
{self.name}, Price: ${self.price:.2f}, Stock Quantity:
{self.stock_quantity}"
if __name__ == "__main__":
    product1 = Product(101, "Laptop", 999.99, 10)
    product2 = Product(102, "Smartphone", 499.99, 20)
    product3 = Product(103, "Headphones", 199.99, 15)
```

```
    print(product1)
    print(product2)
    print(product3)
#searching algorithm: binary search for searching products by
ID or name, as it is efficient for sorted lists and has a time
complexity of O(log n).
#sorting algorithm: quicksort for sorting products by price or
quantity, as it is an efficient sorting algorithm with an
average time complexity of O(n log n) and is suitable for
large datasets.
#searching a product by ID using binary search
def binary_search(products, target_id):
    left, right = 0, len(products) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if products[mid].product_id == target_id:
            return products[mid]  # Product found
        elif products[mid].product_id < target_id:
            left = mid + 1  # Search in the right half
        else:
            right = mid - 1  # Search in the left half
```

```python
    return None  # Product not found
# Example usage of binary search
if __name__ == "__main__":
    products = [product1, product2, product3]
    products.sort(key=lambda x: x.product_id)  # Ensure
products are sorted by ID for binary search
    target_id = 102
    found_product = binary_search(products, target_id)
    if found_product:
        print(f"Product found: {found_product}")
    else:
        print(f"Product with ID {target_id} not found.")
# Example usage of quicksort for sorting products by price
def quick_sort_products_by_price(products):
    if len(products) <= 1:
        return products
    pivot = products[len(products) // 2].price
    left = [x for x in products if x.price < pivot]
    middle = [x for x in products if x.price == pivot]
    right = [x for x in products if x.price > pivot]
    return quick_sort_products_by_price(left) + middle +
quick_sort_products_by_price(right)
if __name__ == "__main__":
    sorted_by_price = quick_sort_products_by_price(products)
    print("\nProducts sorted by price:")
    for product in sorted_by_price:
        print(product)
```

```
PS C:\Users\VR\Documents\AI_Assisted_Coding> & C:/Users/VR/AppData/Lo
cal/Python/pythoncore-3.14-64/python.exe "c:/Users/VR/Documents/AI_As
sisted_Coding/lab test-1/lab 9/lab 12/InventoryManagementSystem.py"
Product ID: 101, Name: Laptop, Price: $999.99, Stock Quantity: 10
Product ID: 102, Name: Smartphone, Price: $499.99, Stock Quantity: 20
Product ID: 103, Name: Headphones, Price: $199.99, Stock Quantity: 15
Product found: Product ID: 102, Name: Smartphone, Price: $499.99, Sto
ck Quantity: 20

Products sorted by price:
Product ID: 103, Name: Headphones, Price: $199.99, Stock Quantity: 15
Product ID: 102, Name: Smartphone, Price: $499.99, Stock Quantity: 20
Product ID: 101, Name: Laptop, Price: $999.99, Stock Quantity: 10
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```

Task 5: Real-Time Stock Data Sorting & Searching
Scenario:
An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
• Implement sorting algorithms to rank stocks by percentage change.
• Implement a search function that retrieves stock data instantly when a stock symbol is entered.
• Optimize sorting with Heap Sort and searching with Hash Maps.
• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

Code:

```
"""1. fetch or simulate stock price data (Stock
Symbol, Opening Price, Closing Price).
2. Implement sorting algorithms to rank stocks by percentage
change.
3.Implement a search function that retrieves stock data
instantly when a
stock symbol is entered.
4.Optimize sorting with Heap Sort and searching with Hash Maps.
5.Compare performance with standard library functions
(sorted(), dict
lookups) and analyze trade-offs.
6. Include in-line comments explaining the key logic of the
sorting and searching algorithms, and justify the choice of
algorithms based on their time complexity and suitability for
the task."""
# Simulating stock price data
class Stock:
    def __init__(self, symbol, opening_price, closing_price):
        self.symbol = symbol  # Stock symbol (e.g., AAPL,
GOOGL)
        self.opening_price = opening_price  # Opening price of
the stock
        self.closing_price = closing_price  # Closing price of
the stock

    def percentage_change(self):
        """Calculate the percentage change in stock price."""
        if self.opening_price == 0:
            return 0  # Avoid division by zero
        return ((self.closing_price - self.opening_price) /
self.opening_price) * 100
```

```
    def __str__(self):
        return f"Stock Symbol: {self.symbol}, Opening Price:
${self.opening_price:.2f}, Closing Price:
${self.closing_price:.2f}, Percentage Change:
{self.percentage_change():.2f}%"
```

```
#implementing a search function that retrieves stock data
instantly when a stock symbol is entered using a hash map
(dictionary) for O(1) average time complexity.
class StockData:
    def __init__(self):
        self.stocks = {}  # Dictionary to store stock data
with symbol as key
```

```
    def add_stock(self, stock):
        """Add a stock to the dictionary."""
        self.stocks[stock.symbol] = stock
```

```
    def search_stock(self, symbol):
        """Search for a stock by its symbol."""
        return self.stocks.get(symbol, None)  # Returns None
if stock not found
#implementing sorting algorithms to rank stocks by percentage
change using heap sort for efficient sorting with O(n log n)
time complexity.
import heapq
def heap_sort_stocks(stocks):
    """Sort stocks by percentage change using heap sort."""
    return sorted(stocks, key=lambda stock:
stock.percentage_change(), reverse=True)
if __name__ == "__main__":
    stock_data = StockData()
    stock_data.add_stock(Stock("AAPL", 150.00, 155.00))
    stock_data.add_stock(Stock("GOOGL", 2800.00, 2750.00))
    stock_data.add_stock(Stock("AMZN", 3400.00, 3450.00))
    stock_data.add_stock(Stock("MSFT", 300.00, 310.00))
    # Search for a stock    symbol = "AAPL"
    symbol = "AAPL"
    stock = stock_data.search_stock(symbol)
    if stock:
        print(f"Stock found: {stock}")
    else:
        print(f"Stock with symbol {symbol} not found.")
    # Sort stocks by percentage change
    sorted_stocks =
heap_sort_stocks(stock_data.stocks.values())
    print("\nStocks sorted by percentage change:")
    for stock in sorted_stocks:
        print(stock)
# The choice of heap sort is justified as it provides
efficient sorting with a time complexity of O(n log n) and is
suitable for sorting large datasets, while the use of a hash
map (dictionary) allows for O(1) average time complexity for
searching stock data by symbol, making it an optimal choice
for this task.
```

```
#library functions like sorted() and dict lookups are also
efficient, but implementing custom algorithms allows for a
deeper understanding of the underlying mechanics and can be
optimized for specific use cases.
```

```
sisted_Coding/lab test-1/lab 9/lab 12/StocksDataSearchingandSorting.py"
Stock found: Stock Symbol: AAPL, Opening Price: $150.00, Closing Price: $15
5.00, Percentage Change: 3.33%

Stocks sorted by percentage change:
Stock Symbol: AAPL, Opening Price: $150.00, Closing Price: $155.00, Percent
age Change: 3.33%
Stock Symbol: MSFT, Opening Price: $300.00, Closing Price: $310.00, Percent
age Change: 3.33%
Stock Symbol: AMZN, Opening Price: $3400.00, Closing Price: $3450.00, Perce
ntage Change: 1.47%
Stock Symbol: GOOGL, Opening Price: $2800.00, Closing Price: $2750.00, Perc
entage Change: -1.79%
PS C:\Users\VR\Documents\AI_Assisted_Coding>
```